

STUDENT NAME	STUDENT ID	HOST CPU / SOC MODEL	OPERATING SYSTEM & KERNEL
Yengsebek Zhannur	240103149	Apple M3	macOS

Generative AI & Tooling Policy: You are explicitly authorized to use LLMs (ChatGPT, Claude, GitHub Copilot) to generate code scripts in any language (C, C++, Python, Go, Rust, Java). However: All execution timings, memory counters, and hardware cache metrics must be physically benchmarked on your host machine. Fabricated numbers receive zero credit.

TASK 1: Host Silicon Audit & Flynn's Taxonomy Mapping

15 Minutes | 20 Points

Query your operating system and CPU registers to document physical core topology, cache boundaries, and vector math extensions.

Terminal Audit Commands: Linux: lscpu && lscpu --caches | macOS: sysctl -a | grep machdep.cpu | Windows: Run wmic cpu get /format:list and inspect Task Manager > Performance > CPU.

Hardware Property	Local Measured Value	Hardware Property	Local Measured Value
CPU Model & Architecture	Apple M3	Base & Max Boost Clock	~2.75 GHz / 4.05 GHz
Physical Hardware Cores	8 (4P + 4E)	Logical Cores (SMT/Threads)	8
L1 Data / Instruction Cache	L1d: 128 KB L1i: 196 KB	L2 Cache (Per Core / Total)	P: 16 MB E: 4 MB (Total 20 MB)
L3 Shared Last-Level Cache	N/A (8 MB System Level Cache)	SIMD Vector Extensions	ARM NEON, AMX, FEAT_DotProd

Q1.1: Map your host machine's execution modes into Flynn's Taxonomy:

My host Apple M3 architecture operates across Flynn's Taxonomy depending on the workload: it functions as SISD during single-threaded sequential execution (such as basic OS logic or single-pass compilation) where one CPU core processes a single instruction stream on one data stream; it utilizes SIMD through ARM NEON vector extensions and hardware dot-product acceleration (FEAT_DotProd) to execute a single vector instruction across multiple data elements simultaneously for matrix and signal processing; and it manifests as MIMD during concurrent OS multi-threading, where independent execution threads run across the 8 physical cores (4 Performance and 4 Efficiency cores) executing distinct instruction streams on separate memory spaces concurrently.

TASK 2: Multi-Thread Scaling Benchmark & Contention Wall

35 Minutes | 30 Points

Write or prompt an AI to generate a compute-bound CPU benchmark parameterized by thread count N. Run 3 iterations per thread count and log wall-clock execution time.

Workload Algorithm: [e.g., Prime Sieve to 5,000,000 / Matrix Multiply]				Language & Runtime: [e.g., C OpenMP / Python multiprocessing / Go]		
Threads (N)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Avg Time TN (s)	Speedup SN = T1/TN	Efficiency EN = SN/N
N = 1 (Baseline)	0.39	0.31	0.30	0.33	1.00x	100.0%
N = 2	0.34	0.34	0.34	0.34	0.97x	48.5%
N = 4	0.41	0.41	0.39	0.40	0.83x	20.8%
N = 8	0.68	0.61	0.61	0.63	0.52x	6.5%
N = 16	1.40	1.31	1.25	1.32	0.25x	1.6%
N = 32	2.85	3.09	2.70	2.88	0.11x	0.3%

Q2.1: Scaling Saturation & Hardware Contention Analysis:

Speedup saturated immediately at \$N=2\$ (\$S_2 = 0.97\times\$) and sharply reversed as thread counts increased. On this Apple M3 host, this early saturation occurs because Python's process creation overhead via multiprocessing (memory space duplication and IPC serialization) exceeds the compute density of the short workload (~0.33s). Furthermore, the heterogeneous core topology (4 Performance + 4 Efficiency cores) creates hardware bottlenecking: when threads scale beyond 4, tasks are scheduled onto slower E-cores and trigger CPU memory bandwidth contention alongside OS context-switching overhead across the 8 physical cores, causing execution times to degrade substantially.

Activity monitor

→

```
CPU 6 frequency: 2058 MHz
CPU 7 frequency: 2013 MHz
(base) zhannur@MacBook-Zhannur ~ % cd ~/Downloads
      chmod +x run_task2.sh
      ./run_task2.sh
chmod: run_task2.sh: No such file or directory
zsh: no such file or directory: ./run_task2.sh
(base) zhannur@MacBook-Zhannur Downloads % nano benchmark.py
(base) zhannur@MacBook-Zhannur Downloads % python3 benchmark.py
Threads: 1 | Time: 0.39 s
Threads: 1 | Time: 0.31 s
Threads: 1 | Time: 0.30 s
Threads: 2 | Time: 0.34 s
Threads: 2 | Time: 0.34 s
Threads: 2 | Time: 0.34 s
Threads: 4 | Time: 0.41 s
Threads: 4 | Time: 0.41 s
Threads: 4 | Time: 0.39 s
Threads: 8 | Time: 0.68 s
Threads: 8 | Time: 0.61 s
Threads: 8 | Time: 0.61 s
Threads: 16 | Time: 1.40 s
Threads: 16 | Time: 1.31 s
Threads: 16 | Time: 1.25 s
Threads: 32 | Time: 2.85 s
Threads: 32 | Time: 3.09 s
Threads: 32 | Time: 2.70 s
(base) zhannur@MacBook-Zhannur Downloads %
```

TASK 3: The Unsynchronized Shared Counter & Race Condition Trap

25 Minutes | 25 Points

Spawn 10 parallel threads. Each thread executes a loop incrementing a shared global integer 1,000,000 times without atomic operations or locks. Theoretical expected total: $10 \times 1,000,000 = 10,000,000$. Execute 10 consecutive runs.

Run	Measured Output	Error (10^7 - Act)	Run	Measured Output	Error (10^7 - Act)
#1	1,937,502	8,062,498	#6	2,046,222	7,953,778
#2	2,028,824	7,971,176	#7	2,398,168	7,601,832
#3	2,371,965	7,628,035	#8	2,062,636	7,937,364
#4	2,403,370	7,596,630	#9	2,096,945	7,903,055
#5	2,007,350	7,992,650	#10	1,957,147	8,042,853

Q3.1: Read-Modify-Write (RMW) Window

Value corruption occurs because the C-statement counter++ decomposes at the machine assembly level into three distinct, non-atomic instructions: Load (read variable from RAM/L3 into register), Add (increment register value by 1), and Store (write register value back to RAM). When multiple CPU cores execute this sequence concurrently without isolation, their execution windows interleave. Core A and Core B read the same initial value simultaneously, increment it locally, and overwrite each other's updates during the Store phase, resulting in lost updates and severe data corruption.

Q3.2: The Synchronization Penalty

Locked execution is substantially slower (~17× slower in this test) because a Mutex enforces complete serialization on shared memory, effectively neutralizing parallel performance. Threads are forced to execute sequentially, spending significant cycles waiting in lock contention queues. Furthermore, acquiring and releasing locks introduces thread-context switching overhead, cache line invalidation across cores, and memory barrier instructions that stall processor pipelines.

TASK 4: Empirical Amdahl Verification vs. Gustafson Scaling Horizon

15 Minutes | 15 Points

Use your empirical single-thread time (T1) and two-thread time (T2) from Task 2 to derive the parallel fraction p of your workload, then evaluate theoretical performance bounds.

1. Parallel Fraction Derivation (p) Formula: $p = 2 \times (T1 - T2) / T1$ T1 = <u>0.33</u> s T2 = <u>0.34</u> s Calculated p = <u>-0.06</u> (decimal between 0.0 and 1.0)	2. Amdahl Theoretical Speedup Ceiling Formula: $S_{max} = 1 / (1 - p)$ Sequential fraction (1 - p) = <u>1.00</u> Max Theoretical Speedup ($N \rightarrow \infty$) = <u>1.00×</u> ×
3. Amdahl 64-Core Projection Formula: $S_{64} = 1 / [(1 - p) + (p / 64)]$ Projected Speedup on 64 Cores = <u>1.00×</u> ×	4. Gustafson Scaled Speedup (Weak Scaling) Formula: $S_{Gustafson} = (1 - p) + (p \times 64)$ Projected Scaled Speedup (64× Problem) = <u>1.00×</u> ×

Q4.1: Strong vs. Weak Scaling Analysis:

Amdahl’s Law assumes a fixed problem size (Strong Scaling), meaning the serial fraction (1 - p) acts as an immutable bottleneck. As core counts increase toward infinity, the execution time becomes entirely dominated by the non-parallelizable code segment, causing theoretical speedup to hard-saturate at 1/(1-p). Conversely, Gustafson’s Law evaluates Weak Scaling, where the problem size scales proportionally alongside core count. Under Gustafson's model, the workload assigned to parallel cores grows while the serial execution overhead remains constant in time, rendering the sequential bottleneck negligible and predicting near-linear speedup approx on a 64-core system.

Submission Guidelines & Deliverables Checklist:

- Complete all 4 tables and technical reflection prompts directly on this document (or submit as an annotated Word / PDF).
- Attach a single terminal screenshot verifying Task 2 executing across active threads (e.g., htop, top, or Task Manager CPU pane).
- Upload the final .docx / PDF to the course portal before the conclusion of the 90-minute lab period.